

maintenant_

Monitor everything. **Manage nothing.**

Un seul container. Ton stack entier monitoré en quelques secondes.

Containers, endpoints, certificats, métriques, status page — tout au même endroit.

STACK

Go · Vue 3 · SQLite

RUNTIME

Docker · K8s · Swarm

FOOTPRINT

~17 MB RAM · 1 binaire

LICENCE

AGPL-3.0 / Commercial

Tu jongles avec 5 outils pour monitorer ton homelab.

Un pour les containers, un pour l'uptime, un pour les certificats, un pour les métriques, un pour la status page. Trop.

AVANT

Portainer

Uptime Kuma

Dozzle

SSL checker

Grafana



APRÈS



maintenant

Un binaire Go. SQLite embarqué. Frontend Vue dans embed.FS.

Docker, Swarm et Kubernetes auto-détectés. Zéro config pour démarrer.

Installé en 30 secondes, tout ton stack visible immédiatement.

17 MB

RAM idle

1

binaire

0

dépendance externe

30 s

pour démarrer

Tout ce que tu monitorais. Enfin ensemble.

Neuf domaines, une seule app. Déclaratif via labels Docker, zero-config pour l'essentiel.

Containers

Auto-discovery Docker & K8s, état, health, restart loops, logs live.

Endpoints

Checks HTTP/TCP via labels. Thresholds, uptime 90j, sparklines.

Heartbeats

Monitoring de cron jobs. URL unique, curl, deadline missed → alerte.

Certificats TLS

Auto-détection depuis les HTTPS. Alertes 30/14/7/3/1 jours.

Ressources

CPU, mémoire, I/O réseau & disque. Temps réel + historique.

Sécurité réseau

Ports 0.0.0.0, DB exposées, host-network, privileged, NodePort.

Update intel

Scan OCI registries. Comparaison de digests. Commandes compose-aware.

Alerting

Moteur unifié, silence rules, retry backoff. Webhook + Discord.

Status page

Page publique, groupes de composants, SSE live, sévérité agrégée.

Auto-discovery Docker & Kubernetes.

Chaque container apparaît à la seconde où il démarre. Aucun fichier de config à maintenir.

✓ Zero-config discovery

Détection via socket Docker monté en read-only, ou via service account K8s.

✓ Health, restarts, logs

Transitions d'état, restart loops, streaming stdout/stderr démuxé.

✓ Compose-aware

Les projets Compose sont regroupés automatiquement.

✓ Kubernetes natif

Deployments, DaemonSets, StatefulSets comme citoyens de première classe.

✓ Runtime auto-déecté

K8s → Swarm manager → Docker. MAINTENANT_RUNTIME pour forcer.

```
docker-compose.yml

services:
  maintenant:
    image: ghcr.io/kolapsis/maintenant:latest
    ports: ["8080:8080"]
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - /proc:/host/proc:ro
      - maintenant-data:/data
    environment:
      MAINTENANT_ADDR: "0.0.0.0:8080"
      MAINTENANT_DB:  "/data/maintenant.db"

# $ docker compose up -d
```

Uptime et cron monitoring, déclarés en labels.

Tu définis tes checks là où vivent tes services. Pas de YAML séparé.

✓ HTTP & TCP

Déclaratif via labels Docker. Simple ou indexé pour plusieurs endpoints.

✓ Seuils configurables

Failure & recovery thresholds, timeouts, méthodes, headers, status attendus.

✓ Historique 90 jours

Response times, uptime history, sparklines visuelles.

✓ Heartbeat pings

URL unique par job. Un curl dans ton cron, maintenant.dev suit le reste.

✓ Deadline missed

Start, finish, durée, exit code. Alerte si le job rate sa fenêtre.



endpoints & heartbeat

```
# Endpoint HTTP via label
labels:
  maintenant.endpoint.http: "https://api/health"
  maintenant.endpoint.interval: "15s"
  maintenant.endpoint.failure-threshold: "3"

# Plusieurs endpoints par container
  maintenant.endpoint.0.http: "https://app/health"
  maintenant.endpoint.1.tcp: "redis:6379"

# Heartbeat – un curl dans ton crontab
curl -fsS -o /dev/null \
  https://now.example.com/ping/{uuid}/$?
```

TLS, ressources, sécurité réseau.

Les trois trous noirs du self-hosting, couverts par défaut — sans agent à installer.



TLS / Certificats

Auto-détection depuis tes endpoints HTTPS.
Monitors standalone pour n'importe quel domaine. Validation de chaîne complète. Alertes à 30, 14, 7, 3 et 1 jour avant expiration.

Community : 5 monitors standalone ·
HTTPS auto-déTECTÉS illimités



Ressources

CPU, mémoire, I/O réseau et disque par container. Graphs de 1 heure à 30 jours. Seuils d'alerte par container avec debounce. Vue 'top consumers' pour triage immédiat.

Community : métriques live · Pro :
historique étendu



Sécurité réseau

Détection automatique : ports bind 0.0.0.0, DB exposées, host-network, privileged, NodePort sans NetworkPolicy. Mapping image → écosystème logiciel via manifest OCI.

Community : insights bruts · Pro : CVE
+ risk score + changelogs

Updates & alerting unifié.

Arrête de faire docker pull à l'aveugle. Arrête de maintenir 3 moteurs d'alerte.



Update intelligence

- ✓ Scan des registries OCI en HEAD-only — aucune image téléchargée.
- ✓ Comparaison de digests pour détecter les updates disponibles.
- ✓ Commandes update & rollback compose-aware avec --project-directory.
- ✓ Intervalle de scan configurable (MAINTENANT_UPDATE_INTERVAL).



Moteur d'alertes

Source	Événements	Sévérité
Container	restart_loop, health_unhealthy	Warning
Endpoint	consecutive_failure	Critical
Heartbeat	deadline_missed	Critical
Certificat	expiring, chain_invalid	Critical
Ressource	cpu / memory threshold	Warning
Update	available	Info

Community : Webhook + Discord · Pro : Slack, Teams, Email + escalation

Status page publique. API REST. Serveur MCP.

Ce que tu monitores est lisible par tes utilisateurs, par tes scripts, et par ton assistant IA.



Status page

Page publique propre, temps réel via SSE.
Groupes de composants, sévérité agrégée sur
tous les monitors. Pas besoin d'un second
outil.

`/status/`



REST API v1

Tout ce que l'UI expose est disponible en API.
Containers, endpoints, heartbeats, certificats,
alertes, webhooks, security, updates.

`/api/v1/`



MCP server

Model Context Protocol intégré. Requête ton
infra, lis tes logs, vérifie tes alertes depuis
n'importe quel client MCP. studio & HTTP avec
OAuth2.

`/mcp`

Community, complet. Pro, pour aller plus loin.

Pas de fonctionnalité clé paywalled. Le Pro ajoute l'alerting avancé, les CVE, les incidents.



Community

Gratuit · AGPL-3.0

- ✓ Container auto-discovery (illimité)
- ✓ Endpoints HTTP/TCP
- ✓ Heartbeat / cron (10 monitors)
- ✓ Certificats TLS (5 + auto illimités)
- ✓ Métriques ressources (live)
- ✓ Network security insights
- ✓ Update intelligence (digest scan)
- ✓ Moteur d'alertes + silence rules
- ✓ Webhook + Discord
- ✓ Status page (1 groupe, 3 composants)
- ✓ REST API + SSE + MCP + PWA
- ✓ Support Docker · Swarm · K8s



Pro

9 € / mois · 90 € / an

- ✓ *Tout ce qu'il y a dans Community*
- ✓ **Canaux Slack, Microsoft Teams, Email**
- ✓ Alert escalation + routing par entité
- ✓ Fenêtres de maintenance
- ✓ Gestion d'incidents (CRUD complet)
- ✓ Status page complète + notifications
- ✓ Historique ressources étendu
- ✓ **CVE enrichment + risk scoring**
- ✓ Changelogs automatiques (OSV.dev)
- ✓ Security posture dashboard
- ✓ Dashboards Swarm / K8s avancés
- ✓ Heartbeats illimités

Un binaire. Une base SQLite. Rien d'autre.

Pas de Redis, pas de Postgres, pas de file d'attente. Volontairement.

Single Go binary · ghcr.io/kolapsis/maintenant

Vue 3 + TypeScript + Tailwind — embedded via embed.FS

Real-time SSE · uPlot charts · PWA support

REST API v1 · SSE broker · MCP server (stdio + HTTP)

Docker runtime

socket.sock:ro

Kubernetes runtime

in-cluster RBAC

SQLite · WAL mode · single-writer · zero external dependencies

Pas d'auth intégré. Par conception.

Comme Prometheus, Dozzle et la plupart des outils self-hostés, maintenant s'assoit derrière ton reverse proxy + auth middleware. Zéro compte utilisateur à gérer en plus.



- ✓ read-only access: maintenant n'intervient jamais sur tes stacks.
- ✓ /ping/{uuid} et /status/ sont publics par design — configure ton proxy en conséquence.
- ✓ Docker socket monté en :ro. /proc monté en :ro. tmpfs sur /tmp. no-new-privileges.

Ce que ça pèse, concrètement.

Assez léger pour un Raspberry Pi. Assez sérieux pour ton cluster.

~17 MB

RAM au repos

1

binaire à déployer

0

dépendance externe

9+

domaines couverts

30 s

pour démarrer

90 j

d'historique uptime

Monitor everything. Manage nothing.

ESSAYER

`docker compose up -d`

SITE

`maintenant.dev`

CODE SOURCE

`github.com/kolapsis/
maintenant`

LICENCE

AGPL-3.0 / Commercial